GRADUATION PROJECT PROPOSAL

Software-Defined Industrial Visual Anomaly Detection System

Cloud MLOps Pipeline & Simulation

Item	Details
Scope	100% software: Python camera & line simulator, AWS serverless MLOps, web dashboard
Models	YOLOv8 + ONNX Runtime (production); PatchCore (offline baseline only)
Datasets	MVTec AD, NEU, DAGM, plus a public Kaggle / Roboflow set for ceramics
Cloud stack	S3, Lambda, SageMaker Serverless Inference, SageMaker Model Registry, EventBridge, DynamoDB, CloudWatch, SNS

1. Project Overview

Manufacturers of ceramic tiles and sanitaryware and of machined metal parts still rely largely on manual visual inspection at the end of the line. Human inspectors tire, vary in judgement and cannot sustain modern line throughput. Hairline cracks, scratches, chips, pits and inclusions are small, low-contrast and often reflective, so defective units regularly reach packaging.

This project designs and implements a computer vision system that inspects every image, localises defects with bounding boxes and confidence scores, and returns a PASS/FAIL verdict in under one second (warm path). The production line is simulated: a Python Camera & Line Simulator streams image events to Amazon S3, which triggers an event-driven AWS serverless pipeline. Results appear in a live web dashboard. No physical hardware or factory access is required.

1.1 Objectives

- Detect and localise surface defects on two material families (ceramics and metals) with high recall.
- Return bounding boxes, class label and confidence per finding, plus an overall PASS/FAIL decision.
- Achieve end-to-end warm latency below 1 second per image using ONNX Runtime; report cold-start latency separately.
- Build a fully serverless, event-driven AWS pipeline with traceability of every decision.
- Provide a Python simulator reproducing realistic line behaviour (rate, batches, bursts, defect mix).
- Deliver an interactive web dashboard for monitoring and for the graduation defense.
- Establish a repeatable MLOps loop: drift detection, retraining, quality gate, registry, gated deployment with rollback.

1.2 Scope and Target KPIs

KPI	Target	Notes
Image-level recall (critical classes)	>= 95%	Primary KPI. An image with any real defect must be FAILed. Box-level recall is reported as a secondary metric.

KPI	Target	Notes
mAP@0.5 (held-out test set)	>= 0.90	Per material and overall. Also reported by defect size (small / medium / large) because tiny defects are the hardest.
False reject rate	<= 5%	Tunable via per-material confidence threshold.
End-to-end latency, warm	< 1.0 s	S3 event to DynamoDB write; p50 / p95 / p99 in CloudWatch.
Cold-start latency	Measured and reported	Reported as a separate KPI. Mitigated to near-warm on defense day (Section 3.5).
Audit coverage	100%	Every image and verdict logged with model version, dataset version and preprocessing version.

Targets are planning values confirmed against the held-out test split and simulated load tests in Milestone 3. Because the data is public rather than factory-captured, results demonstrate the method and pipeline, not guaranteed performance on a specific production line.

2. Datasets and Data Preparation

2.1 Benchmark Dataset: MVTec AD

MVTec AD is the standard industrial anomaly-detection benchmark: 15 categories, defect-free training images and test images with pixel-accurate masks. Relevant categories: tile (ceramic-like), metal_nut, screw and zipper (metal). It serves as an objective benchmark for PatchCore versus YOLOv8 and as a source of training data. MVTec AD is released for non-commercial research, which fits an academic project.

2.2 Multi-Source Online Dataset Strategy (via Roboflow)

Material	Public sources	Typical defects
Ceramics	MVTec AD (tile) + one public Kaggle surface / tile defect dataset	Crack, chip, stain, surface bump, glaze defect
Metals	MVTec AD (metal_nut, screw, zipper) + NEU Surface Defect Database + DAGM texture defects	Scratch, dent, pitting, inclusion, crazing, rolled-in scale

Item	Plan
Source selection	Shortlist in Milestone 1 by access, licence, annotations and image quality. Record name, URL, licence and image count in the dataset register.
Unified classes	crack, scratch, chip, pit, stain/rust (final list fixed in M1; source labels mapped to these).
Volume target	>= 300 images per material, including >= 80 defective samples per material; counts are confirmed once the sources are downloaded.
Negative samples	Every source must contribute defect-free images alongside defective ones, so the model cannot learn source identity (e.g. 'grayscale image = metal = defect').
Quality control	Second-reviewer check on 20% of manually labelled images; automated duplicate and near-duplicate removal across sources.
Split	70% train / 20% validation / 10% test, split by source-image group to avoid leakage.
Augmentation	Flip, rotation, brightness/contrast, blur, noise, mosaic. Training set only.
Versioning	Every Roboflow dataset version is tagged and linked to the model version, the preprocessing version and the git commit.

Known limitation: public datasets differ in resolution, lighting and defect style, and several are lab-captured. The project addresses this through Section 2.3, per-source evaluation, and an explicit

domain-gap discussion in the final report. Glass is outside the scope of this version and is listed as future work; the material-profile design allows a glass profile to be added without changing the architecture.

2.3 Image Normalisation and Preprocessing

Resizing alone is not enough: MVTec AD images are up to 1024 px colour, NEU images are 200x200 grayscale, and DAGM images are grayscale textures. The project therefore uses one shared preprocessing module, `preprocess.py`, in three layers. The layers are applied in the order B, C, A: colour unification and the material profile run at the native resolution and the letterbox comes last, so that profile statistics are not distorted by padding and hairline defects are enhanced before any resampling.

Layer	What is applied	Why
A. Geometry (all images)	Letterbox to 640x640 (keep aspect ratio, pad with a constant grey value). Upscaling uses Lanczos. Applied last, after layers B and C. Small sources (NEU) are upscaled; large sources (MVTec) are tested at `imgsz` 640 versus 960 in M2.	Stretching distorts thin cracks. Downscaling 1024 px images to 640 px can erase hairline cracks and pinholes.
B. Colour unification (all images)	Convert every image to grayscale, then replicate to 3 channels. Colour is kept in one ablation variant (see Rules below) because the stain/rust class may depend on it.	Removes the colour-versus-source shortcut so all sources look alike to the network.
C. Material profile	Ceramics: flat-field / background subtraction (uneven illumination, glaze gloss). Metals: CLAHE on the luminance channel (low-contrast scratches).	Each material has a different dominant nuisance. The profile is selected by the material label (the simulator sends it as S3 metadata) and stored as YAML parameters. No profile uses a blur or smoothing filter, because it would erase hairline defects.

Rules that keep training and serving consistent

- **One module, two callers.** The same `preprocess.py` (pinned by version tag) is imported by the training notebook and packaged in the Lambda layer. This removes train/serve skew.
- **Lossless or near-lossless storage.** Images are stored and sent as PNG or JPEG quality >= 95. Heavy JPEG compression removes hairline cracks, so lossy re-encoding is avoided.
- **Roboflow for labelling and versioning only.** preprocess.py is run on the images before training, and Roboflow's own preprocessing is limited to auto-orient. Images are therefore never processed twice, and Roboflow cannot apply a different profile than the Lambda does.
- **Pixel scaling once.** The 0-1 scaling is done in the YOLO / ONNX input stage and is identical in training and serving; no per-source mean/std normalisation is applied.
- **Preprocessing version in every record.** The DynamoDB record and the model package metadata store the preprocessing version.
- **Per-source evaluation.** Results are reported per source dataset and per material, not only as one merged number, to expose domain-gap problems.
- **Ablation in M2 (evidence for the defense).** Compare (i) letterbox only, (ii) A+B+C, and (iii) A+C with colour retained, to check the stain/rust class, all on the same split, reporting mAP, small-defect mAP and image-level recall.

2.4 Mask-to-Bounding-Box Conversion

Box quality directly affects YOLO training, and tiny defects are the most sensitive. The conversion is implemented as one script, `mask_to_boxes.py`, with unit tests. Its parameters and version are recorded in the dataset register so every dataset version is reproducible.

The conversion runs at the native image resolution. Boxes are then mapped to the 640x640 letterboxed frame with the same transform as the image (`transform_boxes`), and the minimum-size rule (step 6) is enforced in that final frame, so that it is expressed in the pixels the model actually sees. Boxes are axis-aligned, as the standard YOLOv8 detection head requires; rotated boxes would need the OBB variant and are not used.

Step	Rule	Default
1. Binarise	Threshold the mask; no aggressive morphological opening, which would delete pinholes.	mask > 0
2. Merge fragments	Morphological closing so fragments of one crack become one region instead of many boxes.	Kernel 5 px (3-7 tested)
3. Components	Connected components with 8-connectivity; one box per component.	8-connectivity
4. Noise filter	Drop components smaller than a minimum area.	Min area 4 px
5. Padding and clipping	Pad each box, then clip to image bounds.	Pad 3 px
6. Minimum size	Expand boxes smaller than the minimum so tiny pinholes stay learnable.	Min side 8 px (in the 640 frame)
7. Long diagonal defects	If aspect > 5 and fill ratio < 0.15, a single box contains mostly background. Variant: split into overlapping sub-boxes along the main axis (aspect <= 3:1). Both variants are compared in M2; the better one is kept.	Compared in M2
8. DAGM ellipses	Convert each ellipse to an axis-aligned box using its rotated extent: w = 2*sqrt(a^2 cos^2(t) + b^2 sin^2(t)), h = 2*sqrt(a^2 sin^2(t) + b^2 cos^2(t)). Flag as 'loose boxes' in the register.	Loose-box flag

Conversion QA:

- Unit tests: single pinhole, fragmented crack, two separate defects, defect touching the image border, empty mask.
- Automated checks: box-area distribution and boxes per image; any box larger than 80% of the image or with an abnormal fill ratio goes to manual review.
- Visual check: boxes drawn on a random 10% sample (in addition to the 20% second-reviewer check) before the first training run.
- Metrics: mAP by size (small / medium / large) and image-level recall are added so tiny-defect failures cannot hide inside an average.

2.5 Tooling

Area	Tools
Data and labelling	MVTec AD, NEU, DAGM, Kaggle, Roboflow, Amazon S3
Modelling	Python, PyTorch, Ultralytics YOLOv8, Anomalib (PatchCore, offline), OpenCV, ONNX / ONNX Runtime
Training and MLOps	SageMaker Training (GPU, Spot) or Colab/Kaggle GPU for low-cost runs; SageMaker Model Registry; experiment tracking
Simulation	Python 3, boto3, Pillow / OpenCV, YAML configuration

Area	Tools
Serving and pipeline	S3, Lambda, SageMaker Serverless Inference, DynamoDB, CloudWatch, EventBridge, SNS
Dashboard	Streamlit; boto3; Plotly / Altair
Delivery	IAM least privilege, deployment scripts (AWS CLI / boto3), Git

3. Model and Performance

3.1 Modelling Strategy

YOLOv8 and PatchCore are both evaluated, but only YOLOv8 is deployed. PatchCore needs a large memory bank of patch embeddings; hosting it on CPU serverless inference risks out-of-memory failures and very slow cold starts. It is therefore used only as an offline research baseline.

	YOLOv8 (supervised)	PatchCore (unsupervised)
Principle	Learns labelled defect classes and predicts boxes directly	Learns normal appearance from defect-free images; flags deviating patches
Output	Bounding box, class, confidence	Anomaly score and heatmap
Strengths	Fast, class-aware, native boxes, small ONNX model	Needs no defect samples; can flag unseen defect types
Limits	Needs labelled defects; cannot detect defect types it was never trained on	Large memory bank; per-category model; weak class information
Role in this project	The only production model (deployed as ONNX on SageMaker Serverless)	Offline M2 baseline only; never deployed

PatchCore baseline protocol (M2)

- Categories: MVTec AD tile, metal_nut, screw, zipper; one model per category, with coreset subsampling.
- Runs on a free Colab / Kaggle GPU. It never touches the AWS endpoint.
- Reported: image-level AUROC, memory-bank size, inference time per image, and a comparison with YOLOv8 on the same categories.
- Honest limitation, stated in the report: YOLOv8 cannot detect defect types absent from its training data. New defect types are handled by the review queue and retraining loop (Section 5), not by a second deployed model.

3.2 Outputs per Image

- Bounding boxes, class label and confidence (0 to 1) for every detection.
- Verdict: PASS if no detection exceeds the material-specific threshold, otherwise FAIL (maximum score recorded).
- Metadata: model version, dataset version, preprocessing version, material, and latency.

3.3 Training and Evaluation

- Fine-tune pretrained YOLOv8n/s on the merged dataset prepared per Sections 2.3-2.4.
- Metrics: precision, recall, mAP@0.5, mAP@0.5:0.95 per material and class; **mAP by defect size**; **image-level recall and false-reject rate**; confusion matrix; per-source results.
- Experiments in M2: preprocessing ablation (Section 2.3), box-conversion variants (Section 2.4), `imgsz` 640 versus 960.
- Threshold tuning on the validation set per material, favouring recall on critical defects.

- Export to ONNX and verify (parity check) that test-set accuracy matches the PyTorch model.
- Error analysis on hard cases (glossy glaze, metallic texture).

3.4 Meeting the Sub-One-Second Latency Target

Stage	Budget (warm)	How it is achieved
Simulator upload + S3 event	50-150 ms	Regional bucket, direct boto3 upload
Lambda preprocessing	50-120 ms	Shared `preprocess.py` (grayscale, material profile, letterbox); lossless/high-quality image
Model inference	150-500 ms	YOLOv8n/s ONNX, ONNX Runtime on CPU, 4-6 GB serverless memory
DynamoDB write	10-30 ms	Single PutItem per image
Total	< 1,000 ms	Measured in CloudWatch (p50 / p95 / p99)

If the preprocessing ablation in M2 shows that a larger `imgsz` is required for small defects, the latency budget is re-validated in M3 before that setting is adopted.

3.5 Cold-Start Strategy for Testing and the Defense

SageMaker Serverless Inference scales to zero. After an idle period the first request can take several seconds, which would damage a live demo. The warm-latency target therefore applies to warm invocations, and cold start is handled with two layers, an optional third layer and a fallback.

Layer	Implementation	When
1. Scheduled keep-warm	EventBridge Scheduler (every 5 min) triggers a small Lambda that invokes the endpoint with a dummy image flagged `warmup=true`. Warm-up requests do not write inspection records and are excluded from KPIs and the dashboard.	Test and defense phases only (cost control); disabled afterwards
2. Pre-demo warm-up	`simulator --warmup N` sends N warm-up images before the demo, in parallel, with N at least equal to the expected concurrency so that several instances are warm; the presenter checks that p95 has stabilised. The dashboard shows an endpoint status indicator (Cold / Warming / Warm) and a Warm-up button.	Before the defense
3. Provisioned Concurrency (optional)	If budget allows, 2 instances for the defense session only. A single ping warms only one instance, so burst demos otherwise rely on the parallel warm-up.	Defense session only; removed afterwards
Fallback	Fixed demo dataset and pre-recorded results (also listed in Risks).	If the live system fails

- In M3 the idle time after which a cold start occurs is measured (5, 10, 15 and 30 minutes) instead of being assumed.
- The CloudWatch metric `ModelSetupTime` is monitored to count and time cold starts.
- Cold-start latency is reported as its own KPI, separate from warm p50 / p95 / p99.
- The inference Lambda has its own cold start (the OpenCV / NumPy layer can add one to three seconds). It is included in the keep-warm schedule, and its cold invocations are counted in the same cold-start KPI.

4. Software Architecture

Three software components; no physical hardware dependency. Flow: **Simulator -> S3 -> Lambda -> SageMaker Serverless (YOLOv8 ONNX) -> DynamoDB**, with a **Streamlit dashboard** reading DynamoDB, S3 (presigned URLs) and CloudWatch.

4.1 Component 1 - Camera & Line Simulator

- Python + boto3. Replays images from the held-out test split, so ground truth is known and live accuracy can be measured.
- Key pattern: `raw-images/{material}/{batch_id}/{image_id}.png` with S3 metadata (material, batch, simulated timestamp).
- Modes: steady stream (1-5 images/s), burst for load tests, scripted demo mode with a chosen defect mix, and `--warmup N`.
- YAML configuration (bucket, rate, defect ratio, material mix); a ground-truth manifest for live recall and false-reject measurement.

4.2 Component 2 - AWS Serverless Pipeline

Service	Role	Key configuration
Amazon S3	Receives images; stores annotated results, datasets, model artefacts, and the review queue	ObjectCreated notification; prefixes per material; `review-queue/` prefix; SSE-KMS; lifecycle rules
AWS Lambda (inference)	Validates file, applies shared preprocessing, invokes endpoint, applies thresholds, writes record, publishes drift metrics	Python; 1 GB; 30 s timeout; DLQ; least-privilege role; preprocessing layer pinned to a version
SageMaker Serverless Endpoint	Hosts the YOLOv8 ONNX model on a prebuilt SageMaker inference container, with a custom inference script (no custom image)	4-6 GB memory; concurrency sized to simulator rate
Amazon DynamoDB	One log item per inspection	On-demand; PK image_id, SK timestamp; GSI on verdict and material
CloudWatch / SNS	Metrics, logs, alarms, e-mail alerts	Latency, error rate, ModelSetupTime, and drift metrics (Section 5.1)
EventBridge	Keep-warm schedule; alarm and registry events that start retraining and deployment	Rules described in Sections 3.5 and 5
SageMaker Model Registry	Versioned registration and approval of each trained model	Section 5.2
Deployment Lambda	Runs on approval: endpoint update, warm-up, smoke test, rollback	Section 5.2

4.3 DynamoDB Log Record

Attribute	Description
image_id (PK), timestamp (SK)	Unique image key and ISO-8601 inspection time
material, batch_id	ceramic / metal; simulated production batch
verdict, max_confidence	PASS or FAIL; highest defect confidence
detections	List of {class, confidence, bbox [x1, y1, x2, y2]}
latency_ms, cold_start	Inference and end-to-end times; flag for a cold invocation
model_version, dataset_version, preprocess_version	Full lineage of the decision
input_stats	Mean brightness and contrast of the input image (used for input-drift monitoring)
s3_uri, ground_truth (optional)	Image location; simulator label for live accuracy

4.4 Component 3 - Web Dashboard

- Live feed with bounding boxes, class, confidence and a PASS/FAIL badge (presigned S3 URLs).
- Analytics: defect rate per material and class, verdicts over time, confidence distribution, filters.
- Performance: latency p50 / p95 / p99, error rate, throughput, separate cold-start panel, and the 1-second marker.
- Model panel: active model, dataset and preprocessing versions, thresholds, live recall / false-reject versus ground truth.
- MLOps panel: drift indicators, review-queue size, last retraining run, gate result, and before/after model comparison.
- Demo controls: simulator start instructions, endpoint status (Cold / Warming / Warm) and Warm-up button.

4.5 Reliability, Security and Cost

- Reliability: S3 event retries, Lambda DLQ and idempotent writes keyed by image_id.
- Security: private buckets, encryption at rest and in transit, per-service IAM roles, no hard-coded credentials, CloudTrail.
- Cost: pay-per-request services; AWS Pricing Calculator estimate in M4; Budgets alarms; teardown scripts. Keep-warm is enabled only for the test and defense phases; free Colab / Kaggle GPUs or Spot instances are used for training and retraining.

5. MLOps Loop: Drift, New Defects and Retraining

This section defines what happens when the model meets defects or conditions it has not seen. The loop is: **detect -> collect -> relabel -> retrain -> gate -> approve -> deploy -> verify (or roll back)**.

Retraining is deliberately human-in-the-loop. Alarms start a review, labelling and model approval are manual decisions, and only training, the quality gate and the deployment workflow are automated.

5.1 Triggers and Monitoring

Lambda publishes custom CloudWatch metrics for every inspection. These metrics and alarms are built in Milestone 4 so that Milestone 5 can use them.

Trigger	Measurement	Action
Defect-rate drift	FAIL rate per material and batch versus baseline (e.g. +/- 3 sigma)	CloudWatch alarm -> SNS e-mail
Confidence drift	Share of detections in the uncertain band (between a low floor, e.g. 0.25, and the material threshold)	Alarm; those images are copied to `review-queue/`
Input drift	Mean brightness / contrast of inputs versus training statistics	Alarm
Performance drop	Live recall and false-reject rate versus simulator ground truth	Alarm
Manual	A new dataset version published in Roboflow	Start the retraining pipeline (an EventBridge event sent by a small trigger script once the dataset version is published)

5.2 Retraining Workflow

The retraining workflow is a **scripted pipeline** (one notebook or script running steps 1-5, versioning every model in the **SageMaker Model Registry**) followed by a deployment Lambda for step 6, started by EventBridge on approval. Training runs on a free Colab / Kaggle GPU or on SageMaker Spot using the same script. The gate and registry metadata are the same as a SageMaker Pipeline would use, so the workflow can be ported to SageMaker Pipelines as future work.

Step	Description
1. Data	Pull the tagged dataset version from Roboflow into S3. Images from `review-queue/` are unlabelled, so they are annotated by hand in Roboflow before the dataset version is created.
2. Train	Fine-tune YOLOv8 using the pinned `preprocess.py` and `mask_to_boxes.py` versions.
3. Evaluate + export	Evaluate on two sets: the previous test set (regression check) and an updated test set that also holds out images of any new defect class; export ONNX; run the parity check against PyTorch.
4. Quality gate	Condition step. All must pass: - Critical image-level recall >= 95% and false-reject <= 5% - mAP not worse than the current model beyond a set margin - No material regresses - Recall on any newly added defect class meets the target on its own test images - Latency benchmark within budget
5. Register	Register in Model Registry as `PendingManualApproval` with dataset version, git commit and preprocessing version; send an SNS notification.
6. Deploy	On approval, EventBridge starts the deployment Lambda: create the new endpoint config -> update the endpoint (blue/green, no downtime) -> warm-up, because the new container starts cold -> smoke test on a golden set -> confirm. Automatic rollback to the previous endpoint config, with an SNS notification, if the smoke test fails.

5.3 Defense Scenario: Held-Out Defect Class

To demonstrate honestly how a genuinely new defect is handled, one defect class is withheld from training:

1. Deploy the model trained without the held-out class (for example, `chip` on ceramics).
2. Run the simulator with images of that class: the dashboard shows missed detections, low-confidence detections, a drift alarm and an e-mail alert.
3. Review-queue images are labelled in Roboflow and a new dataset version is created.
4. The pipeline retrains, passes the gate, registers and (after approval) deploys the new model.
5. The dashboard compares before and after: recall on the new class, overall KPIs, and latency.

5.4 Scope Control

The loop is large for a single milestone. To keep it deliverable, it is scripted rather than orchestrated by SageMaker Pipelines: the drift metrics are built in M4, and M5 implements one complete end-to-end run with a nano model. The gate logic and registry metadata do not depend on the orchestrator.

6. Milestone Breakdown

Milestone 1: Requirements, Data Curation and Environment

Objective: Define requirements and build the data and cloud foundations.

Tasks	Deliverables
- Define taxonomy, unified classes, KPIs and simulator requirements - Select public datasets; record licences - Create Roboflow project; import and harmonise sources - Implement and unit-test `mask_to_boxes.py`; visual QA on a 10% sample - Implement `preprocess.py` with material profiles - Provision AWS account, IAM and S3 with deployment scripts;	- Requirements and KPI specification - Dataset register (with conversion parameters) - First versioned Roboflow dataset - `mask_to_boxes.py` and `preprocess.py` with tests - Working AWS environment

Tasks	Deliverables
budget alarms	

Milestone 2: Modelling, Ablations and ONNX Benchmarking

Objective: Train and compare models; select the production YOLOv8 ONNX model.

Tasks	Deliverables
- PatchCore offline baseline on MVTec tile, metal_nut, screw, zipper (AUROC, memory size, time) - Fine-tune YOLOv8 with augmentation - Ablations: preprocessing (letterbox only / A+B+C / colour retained), box-conversion variants, `imgsz` - Per-source and per-material evaluation; mAP by size; image-level recall - Threshold tuning; ONNX export with parity check; CPU latency	- Tracked experiments - Ablation and benchmark report - Selected ONNX model + thresholds - Annotated sample outputs

Milestone 3: Serverless Deployment and Latency

Objective: Deploy the selected model and meet the warm-latency target.

Tasks	Deliverables
- Package the ONNX model with its inference script for the prebuilt SageMaker container; register the model - Deploy the SageMaker Serverless endpoint; tune memory and concurrency - Load-test warm and cold latency; measure idle time to cold start (5/10/15/30 min) - Evaluate keep-warm and parallel warm-up - Confirm final accuracy on the held-out test set	- Model package with version tag - Live endpoint - Latency report: warm p50/p95/p99 and cold start - Final accuracy report

Milestone 4: Simulator, Pipeline and Monitoring

Objective: Build the simulator and connect the event-driven flow, including monitoring needed for MLOps.

Tasks	Deliverables
- Implement the simulator (stream, burst, demo, warm-up, ground-truth manifest) - S3 notifications and Lambda (validation, shared preprocessing, invoke, thresholds) - DynamoDB table, DLQ, error handling - Publish drift metrics; alarms and SNS; `review-queue/` copy logic - EventBridge keep-warm schedule; end-to-end tests; cost estimate and security review	- Simulator with documentation - Deployed pipeline (deployment scripts) - DynamoDB sample records - Drift metrics and alarms - Cost estimate and security review

Milestone 5: Dashboard, MLOps Loop and Defense

Objective: Complete the dashboard, run the retraining loop once end to end, validate the system and prepare the defense.

Tasks	Deliverables
- Build the dashboard (live feed, analytics, performance, model	- KPI acceptance report

Tasks	Deliverables
and MLOps panels) - Scripted retraining workflow: train -> evaluate -> gate -> register; deployment Lambda with warm-up, smoke test and rollback - Run the held-out defect-class scenario and record before/after - Full simulation versus KPI targets - Defense preparation: enable keep-warm, run the warm-up, demo script, fallback dataset - Final report and presentation	- Dashboard and deployment guide - Retraining and versioning runbook - Before/after comparison - Final report, presentation, live demo

7. Summary Mapping Table

Requirement	Solution	Milestone	Outcome
Detect cracks and surface defects	YOLOv8 detector; PatchCore offline baseline	M2	Model meeting image-level recall target
Handle differing sources	Shared `preprocess.py`, negatives per source, per-source evaluation, ablation	M1, M2	Evidence that preprocessing helps
Accurate boxes for tiny defects	Fixed `mask_to_boxes.py` with tests and QA	M1, M2	Reproducible, low-noise labels
No physical data collection	Public datasets via Roboflow	M1, M2	Versioned dataset
Simulate the line	Python + boto3 simulator	M4	Repeatable event stream with ground truth
Latency under 1 s (warm)	ONNX Runtime, right-sized memory	M3	Verified p95 report
Cold start under control	Keep-warm, parallel warm-up script, separate KPI	M3, M5	Smooth defense demo
Automatic trigger	S3 event + Lambda	M4	Hands-free inspection
Traceable decisions	DynamoDB with model, dataset and preprocessing versions	M4	Auditable record
Real-time visibility	Streamlit + CloudWatch	M5	Defense-ready interface
Drift and new defects	Alarms, review queue, scripted retraining workflow, gate, registry approval, rollback	M4, M5	Complete MLOps lifecycle

Key Risks and Mitigations

Risk	Mitigation
Domain gap between public sources	Shared preprocessing, negatives from every source, per-source evaluation, material thresholds, documented limitations
Tiny defects lost by resizing or by noisy boxes	Higher `imgsz` tested in M2; box-conversion rules with tests and visual QA; mAP by size reported
Too few defect samples for some classes	Combine several sources, augmentation; report limits honestly
Glass not covered	Documented as future work; the material-profile design lets a glass profile be added without changing the architecture
Unseen defect types (YOLO cannot	Review queue and retraining loop; stated as a limitation; demonstrated

Risk	Mitigation
detect them)	by the held-out-class scenario
PatchCore memory / cold-start problems	Not deployed; offline baseline only
Cold start during the defense	Keep-warm, parallel pre-demo warm-up, endpoint status indicator, recorded fallback (optional Provisioned Concurrency)
Train/serve skew	Single versioned preprocessing module shared by training and Lambda; ONNX parity check
Dataset licences	Licence register in M1; non-commercial academic use; cite all sources
AWS cost overrun	Budgets alarms, small model, free Colab / Kaggle GPUs or Spot training, keep-warm only for test/defense phases, teardown scripts
MLOps pipeline too large for M5	Drift metrics ready from M4; one full run with a nano model; the workflow is scripted, not orchestrated by SageMaker Pipelines
Delivery risk	Dashboard built incrementally from M4; fixed demo dataset as defense fallback

8. Executive Conclusion

The system demonstrates a complete, production-style computer vision and Cloud MLOps solution without physical hardware. Public industrial benchmarks curated through Roboflow make the work comparable and reproducible. YOLOv8 provides fast, class-aware detection with boxes and confidence scores; PatchCore serves as an offline benchmark. The design adds the engineering rigour a reviewer expects: consistent preprocessing across heterogeneous sources, reproducible box conversion for tiny defects, a deployment that avoids known serverless pitfalls, and a defined loop for drift and new defect types. The scope is two material families, with glass identified as future work.

- **Cost efficiency:** serverless design billed per inspected image; no idle GPU servers.
- **Speed:** under about one second when warm, with cold start measured, reported and mitigated.
- **Traceability:** each decision stores model, dataset and preprocessing versions.
- **Visibility:** dashboard with bounding boxes, verdicts, CloudWatch metrics and MLOps status.
- **Continuous improvement:** drift triggers, gated retraining, registry approval and rollback.

Next Steps

- Confirm the unified classes, the dataset list and the project scope with the supervisor.
- Create the AWS account, IAM roles and budget alarms; initialise Git and the deployment scripts.
- Download datasets, register licences, create the Roboflow project.
- Implement and test `mask_to_boxes.py` and `preprocess.py` first; they feed everything downstream. Then start Milestone 1.